

PRACTICE EXERCISES OF THE MICROPROCESSORS & MICROCONTROLLERS

Instructor: The Tung Than

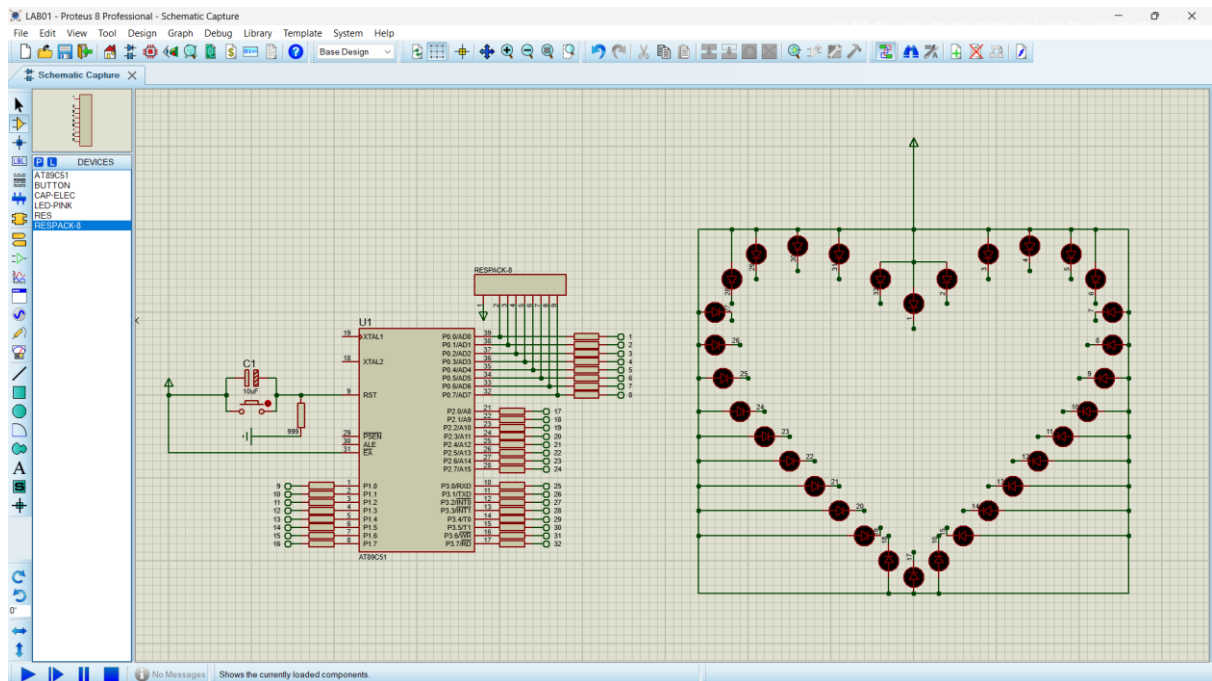
Student's name: Nguyễn Đông Quân

Student code: 24521438

PRACTICE REPORT NO #1: ACQUAINTANCE WITH PROTEUS AND THE 8051 MICROCONTROLLER FAMILY

I. Content 1

Design a heart-led circuit consisting of 32 LEDs controlled by AT89C51.



II. Content 2

Build an Assembly program so that the circuit can run continuously with at least 3 effects, each effect runs within 5 seconds.

❖ Source code:

```
=====
;
; Main.asm file generated by New Project wizard
; Processor: AT89C51
; Compiler: ASEM-51 (Proteus)
=====
```

```
$NOMOD51
```

```
$INCLUDE (8051.MCU)
```

```
=====
;
; RESET VECTOR
;
=====
```

```
    org 0000h
```

```
    jmp Start
```

```
=====
;
; CODE SEGMENT
;
=====
```

```
    org 0100h
```

```
Start:
```

```
    ; Initial State: Turn OFF all LEDs (Logic 1 = OFF)
```

```
    MOV P0, #0FFH
```

```
    MOV P1, #0FFH
```

```
    MOV P2, #0FFH
```

```
    MOV P3, #0FFH
```

```
; Infinite loop for 3 effects
```

```
MainLoop:
```

```
    CALL Eff1_Blink
```

```
    CALL Eff2_Heartbeat
```

```
    CALL Eff3_QuadrantChase
```

```
    JMP MainLoop
```

```
; EFFECT 1: BLINK (5s).
```

```
; Loop 5 times: ON 500ms, OFF 500ms
```

```
Eff1_Blink:
```

```
    MOV R5, #5      ; R5 = loop counter (5 times)
```

```
Loop_E1:
```

```
    ; Turn ON all LEDs (Logic 0 = ON)
```

```
    MOV P0, #00H
```

```
MOV P1, #00H
MOV P2, #00H
MOV P3, #00H
MOV R4, #5      ; R4 = 5 -> Delay 500ms
CALL Delay_R4x100ms
```

```
; Turn OFF all LEDs (Logic 1 = OFF)
```

```
MOV P0, #0FFH
MOV P1, #0FFH
MOV P2, #0FFH
MOV P3, #0FFH
MOV R4, #5      ; Delay 500ms
CALL Delay_R4x100ms
```

```
DJNZ R5, Loop_E1 ; Repeat 5 times
RET
```

```
; EFFECT 2: HEARTBEAT (5s).
```

```
; Double beat: ON 100ms/OFF 100ms -> ON 100ms/OFF 700ms
```

```
Eff2_Heartbeat:
```

```
    MOV R5, #5      ; Loop 5 beats
```

```
Loop_E2:
```

```
    ; Beat 1 (Fast)
    MOV P0, #00H
    MOV P1, #00H
    MOV P2, #00H
    MOV P3, #00H
    MOV R4, #1      ; ON 100ms
    CALL Delay_R4x100ms
```

```
    MOV P0, #0FFH
    MOV P1, #0FFH
    MOV P2, #0FFH
    MOV P3, #0FFH
    MOV R4, #1      ; OFF 100ms
    CALL Delay_R4x100ms
```

```

; Beat 2 (Slow)
MOV P0, #00H
MOV P1, #00H
MOV P2, #00H
MOV P3, #00H
MOV R4, #1      ; ON 100ms
CALL Delay_R4x100ms

MOV P0, #0FFH
MOV P1, #0FFH
MOV P2, #0FFH
MOV P3, #0FFH
MOV R4, #7      ; OFF 700ms (Rest period)
CALL Delay_R4x100ms

DJNZ R5, Loop_E2
RET

```

; EFFECT 3: QUADRANT CHASE (4.8s).

Eff3_QuadrantChase:

```

MOV R5, #6      ; Loop 6 rounds

```

Loop_E3:

```

; Step 1: ON P0, ensure others are OFF

```

```

MOV P3, #0FFH   ; Clear previous step (P3)

```

```

MOV P0, #00H

```

```

MOV R4, #2      ; Delay 200ms

```

```

CALL Delay_R4x100ms

```

```

; Step 2: ON P1, turn OFF P0

```

```

MOV P0, #0FFH

```

```

MOV P1, #00H

```

```

MOV R4, #2

```

```

CALL Delay_R4x100ms

```

```

; Step 3: ON P2, turn OFF P1

```

```

MOV P1, #0FFH

```

```

MOV P2, #00H

```

```
MOV R4, #2
CALL Delay_R4x100ms
```

```
; Step 4: ON P3, turn OFF P2
```

```
MOV P2, #0FFH
```

```
MOV P3, #00H
```

```
MOV R4, #2
```

```
CALL Delay_R4x100ms
```

```
DJNZ R5, Loop_E3
```

```
RET
```

```
; REAL-TIME DELAY 12MHz (1 Machine Cycle = 1µs)
```

```
; Multiplier delay: Time = R4 x 100ms
```

```
Delay_R4x100ms:
```

```
    PUSH 04H      ; Save R4 to Stack to prevent corruption
```

```
D_R4: CALL Delay100ms
```

```
    DJNZ R4, D_R4
```

```
    POP 04H       ; Restore R4 original value
```

```
    RET
```

```
; Standard 100ms delay:  $200 * (248 * 2 + 4) + 13 = 100,013$  cycles (~100ms)
```

```
Delay100ms:
```

```
    PUSH 06H      ; Save R6 to Stack
```

```
    PUSH 07H      ; Save R7 to Stack
```

```
    MOV R6, #200   ; Outer loop
```

```
D1:  MOV R7, #248   ; Inner loop adjusted for 12MHz precision
```

```
D2:  DJNZ R7, D2    ; 2 cycles per loop
```

```
    NOP           ; Timing compensation (1 cycle)
```

```
    DJNZ R6, D1    ; 2 cycles per loop
```

```
    POP 07H       ; Restore R7
```

```
    POP 06H       ; Restore R6
```

```
    RET
```

```
;=====
```

```
END
```

❖ **Giải thích chi tiết:**

- **Vi điều khiển:** AT89C51 sử dụng thạch anh 12MHz (1 chu kỳ máy = 1μs).
- **Cấu trúc Port:** P0, P1, P2, P3 điều khiển 4 cụm LED trên mạch hình trái tim.
- **Logic điều khiển (Active-Low):**
 - **Bit = 0:** Chân Port kéo xuống GND → Đèn sáng.
 - **Bit = 1:** Chân Port kéo lên mức cao (riêng Port 0 ở trạng thái Z được Res Pack kéo lên) → Đèn tắt.

Hiệu ứng 1: Blinking

Hiệu ứng thực hiện chớp tắt đồng loạt tất cả các LED 5 lần với chu kỳ 1 giây (500ms sáng, 500ms tắt).

```
Eff1_Blink:
    MOV R5, #5      ; R5 = loop counter (5 times)
Loop_E1:
    ; Turn ON all LEDs (Logic 0 = ON)
    MOV P0, #00H
    MOV P1, #00H
    MOV P2, #00H
    MOV P3, #00H
    MOV R4, #5      ; R4 = 5 -> Delay 500ms
    CALL Delay_R4x100ms

    ; Turn OFF all LEDs (Logic 1 = OFF)
    MOV P0, #0FFH
    MOV P1, #0FFH
    MOV P2, #0FFH
    MOV P3, #0FFH
    MOV R4, #5      ; Delay 500ms
    CALL Delay_R4x100ms

    DJNZ R5, Loop_E1 ; Repeat 5 times
    RET
```

- **MOV R5, #5:** Khởi tạo số lần lặp lại hiệu ứng chớp tắt là 5 lần.
- **MOV Px, #00H:** Xuất logic 0 ra tất cả các Port 0, 1, 2, 3 làm toàn bộ LED sáng.
- **MOV R4, #5 & CALL Delay_R4x100ms:** Gán giá trị 5 vào thanh ghi R4 để chuẩn bị tạo độ trễ 500ms.
- **CALL Delay_R4x100ms:** Gọi hàm tạo thời gian trễ (5 * 100ms = 500ms).
- **MOV Px, #0FFH:** Xuất mức logic 1 ra các Port làm toàn bộ LED tắt.

- **DJNZ R5, Loop_E1:** Giảm giá trị R5 đi 1 đơn vị, nếu R5 khác 0 thì quay lại nhãn Loop_E1 để tiếp tục nhảy.

Hiệu ứng 2: Heartbeat

Mô phỏng nhịp đập sinh học gồm 2 nhịp nhanh và một quãng nghỉ dài.

Eff2_Heartbeat:

MOV R5, #5 ; Loop 5 beats

Loop_E2:

; Beat 1 (Fast)

MOV P0, #00H

MOV P1, #00H

MOV P2, #00H

MOV P3, #00H

MOV R4, #1 ; ON 100ms

CALL Delay_R4x100ms

MOV P0, #0FFH

MOV P1, #0FFH

MOV P2, #0FFH

MOV P3, #0FFH

MOV R4, #1 ; OFF 100ms

CALL Delay_R4x100ms

; Beat 2 (Slow)

MOV P0, #00H

MOV P1, #00H

MOV P2, #00H

MOV P3, #00H

MOV R4, #1 ; ON 100ms

CALL Delay_R4x100ms

MOV P0, #0FFH

MOV P1, #0FFH

MOV P2, #0FFH

MOV P3, #0FFH

MOV R4, #7 ; OFF 700ms (Rest period)

CALL Delay_R4x100ms

```
DJNZ R5, Loop_E2  
RET
```

- **MOV R5, #5:** Khởi tạo lặp lại hiệu ứng nhịp tim 5 lần.
- **Beat 1 (Fast):** Bật toàn bộ LED trong 100ms (R4=1) và tắt trong 100ms để tạo nhịp đập nhanh đầu tiên.
- **Beat 2 (Slow):** Bật toàn bộ LED trong 100ms và tắt trong 700ms (R4=7) để tạo quãng nghỉ giữa các nhịp đập, mô phỏng tiếng tim đập thực tế.
- **DJNZ R5, Loop_E2:** Lặp lại chu kỳ đập kép cho đến khi đủ 5 lần.

Hiệu ứng 3: Quadrant Chase

Eff3_QuadrantChase:

```
MOV R5, #6 ; Loop 6 rounds
```

Loop_E3:

```
; Step 1: ON P0, ensure others are OFF
```

```
MOV P3, #0FFH ; Clear previous step (P3)
```

```
MOV P0, #00H
```

```
MOV R4, #2 ; Delay 200ms
```

```
CALL Delay_R4x100ms
```

```
; Step 2: ON P1, turn OFF P0
```

```
MOV P0, #0FFH
```

```
MOV P1, #00H
```

```
MOV R4, #2
```

```
CALL Delay_R4x100ms
```

```
; Step 3: ON P2, turn OFF P1
```

```
MOV P1, #0FFH
```

```
MOV P2, #00H
```

```
MOV R4, #2
```

```
CALL Delay_R4x100ms
```

```
; Step 4: ON P3, turn OFF P2
```

```
MOV P2, #0FFH
```

```
MOV P3, #00H
```

```
MOV R4, #2
```

```
CALL Delay_R4x100ms
```



```
DJNZ R5, Loop_E3
RET
```

- **MOV R5, #6:** Khởi tạo vòng lặp Chase 6 lần.
- **MOV P3, #0FFH:** Tắt cụm LED cuối cùng (P3) của vòng lặp trước đó để đảm bảo chỉ có 1 cụm sáng tại một thời điểm.
- **MOV P0, #00H:** Bật cụm LED thuộc Port 0.
- **MOV R4, #2:** Tạo trễ 200ms giúp mắt người thấy hiệu ứng dịch chuyển giữa các cụm LED.
- **MOV Px, #0FFH:** Tắt cụm LED hiện tại trước khi chuyển sang bật cụm LED kế tiếp (P1, P2, P3).
- **DJNZ R5, Loop_E3:** Lặp lại vòng Chase đủ 6 lần trước khi quay về hiệu ứng Blinking ban đầu.

Bộ tạo độ trễ (Delay Utility):

```
; Multiplier delay: Time = R4 x 100ms
Delay_R4x100ms:
    PUSH 04H      ; Save R4 to Stack to prevent corruption
D_R4: CALL Delay100ms
    DJNZ R4, D_R4
    POP 04H       ; Restore R4 original value
    RET

; Standard 100ms delay: 200 * (248 * 2 + 4) + 13 = 100,013 cycles (~100ms)
Delay100ms:
    PUSH 06H      ; Save R6 to Stack
    PUSH 07H      ; Save R7 to Stack
    MOV R6, #200   ; Outer loop
D1:  MOV R7, #248   ; Inner loop adjusted for 12MHz precision
D2:  DJNZ R7, D2    ; 2 cycles per loop
    NOP           ; Timing compensation (1 cycle)
    DJNZ R6, D1    ; 2 cycles per loop
    POP 07H       ; Restore R7
    POP 06H       ; Restore R6
    RET
```

- **Nguyên lý đa tầng:** Sử dụng hàm Delay100ms làm đơn vị gốc, kết hợp với thanh ghi R4 làm hệ số nhân để tạo ra các khoảng thời gian tùy ý (ví dụ: R4=5 tạo trễ 500ms, R4=7 tạo trễ 700ms).

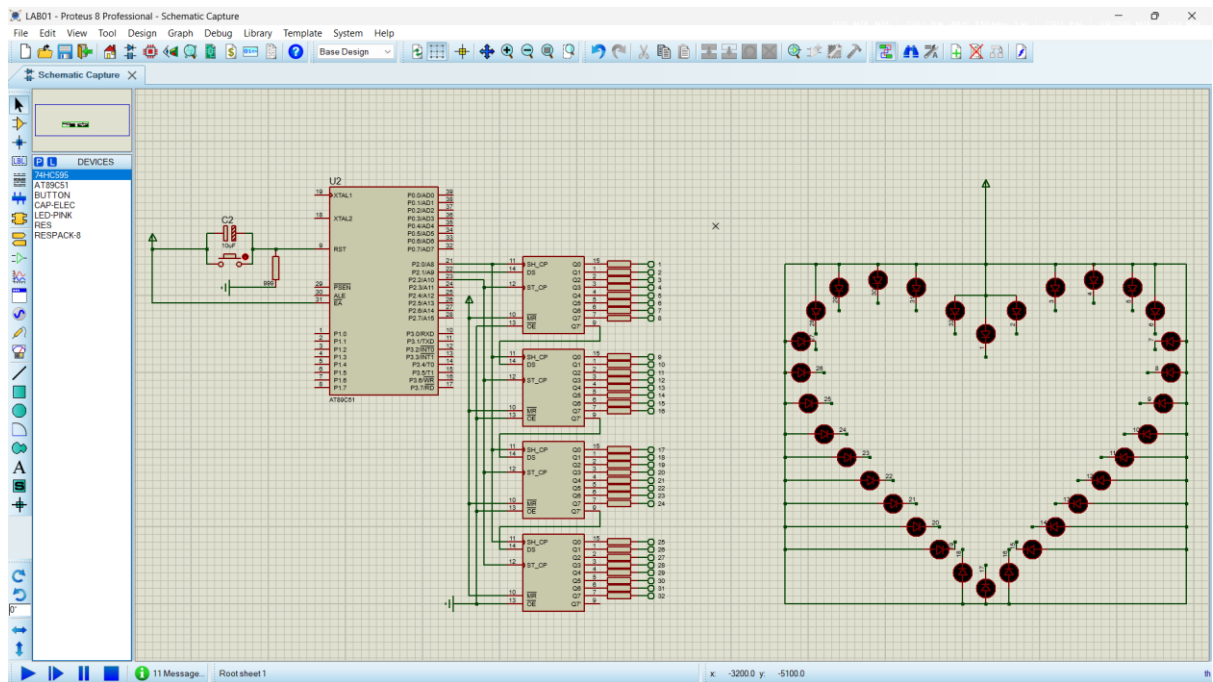
- **Bảo vệ tài nguyên:** Toàn bộ các thanh ghi trung gian (\$R4, R6, R7\$) đều được cất vào Stack trước khi dùng và lấy ra sau khi xong. Việc này giúp hàm Delay chạy độc lập, không làm sai lệch dữ liệu của các chương trình hiệu ứng khác.
- **Tạo độ trễ toán học:**
 - Với thạch anh 12MHz, mỗi lệnh tiêu tốn các chu kỳ máy (1 cycle = 1 μ s).
 - Vòng lặp con D2 chạy 248 lần, kết hợp với các lệnh hỗ trợ (NOP, DJNZ, MOV) tạo thành một vòng lặp lớn (D1) tốn đúng 500 μ s.
 - Lặp lại vòng D1 200 lần giúp tạo ra khoảng trễ xấp xỉ 100,000 μ s (tức 100ms) với sai số cực thấp (0.013%).

III. Exercises

1. From the initial design, it can be observed that directly controlling 32 LEDs would require 32 microcontroller pins, which limits the availability of pins for other functions. Is it possible to implement a circuit that minimizes the number of microcontroller pins while still controlling all 32 LEDs? If possible, propose and implement a solution using suitable components. Draw the schematic circuit and simulate the design using Proteus. Explain the operating principle of the circuit and the method used to control the LED patterns.

Giải pháp tối ưu nhất để giảm thiểu số lượng chân vi điều khiển là sử dụng IC thanh ghi dịch vào nối tiếp - ra song song (SIPO Shift Register), cụ thể là IC 74HC595.

- **Giải pháp đề xuất:** Bằng cách ghép nối tiếp (cascade) 4 IC 74HC595, chúng ta có thể điều khiển 32 LED chỉ bằng 3 chân của vi điều khiển. Chân xuất dữ liệu nối tiếp (Q7') của IC trước được nối trực tiếp vào chân nhận dữ liệu (DS) của IC sau, tạo thành một chuỗi thanh ghi dịch 32-bit liên mạch.
- **Nguyên lý hoạt động & Phương pháp điều khiển:** Vi điều khiển gửi nối tiếp 32 bit dữ liệu qua một chân Data (DS_PIN). Với mỗi bit được đưa ra, chân Shift Clock (SH_CP) sẽ được cấp một xung để đẩy bit đó vào thanh ghi dịch bên trong IC. Khi toàn bộ 32 bit đã nằm đúng vị trí trong 4 IC, chân Latch Clock (ST_CP) sẽ được cấp xung. Thao tác này chuyển dữ liệu từ thanh ghi dịch (Shift Register) sang thanh ghi chốt (Storage Register), làm cập nhật đồng loạt trạng thái của cả 32 LED mà không gây ra hiện tượng nhấp nháy hay bóng ma (ghosting).



❖ Source code:

```

=====
;
; Main.asm file generated by New Project wizard
; Processor: AT89C51
; Compiler: ASEM-51 (Proteus)
=====

$NOMOD51
$INCLUDE (8051.MCU)

=====
;
; 74HC595 PIN DEFINITIONS & VIRTUAL RAM
=====

SH_CP BIT P2.0
DS_PIN BIT P2.1
ST_CP BIT P2.2

LED_P0 DATA 30H
LED_P1 DATA 31H
LED_P2 DATA 32H
LED_P3 DATA 33H

```

```

;=====
; RESET VECTOR
;=====

    org 0000h
    jmp Start

;=====
; CODE SEGMENT
;=====

    org 0100h
Start:
    ; Initial State: Turn OFF all LEDs (Logic 1 = OFF)
    MOV LED_P0, #0FFH
    MOV LED_P1, #0FFH
    MOV LED_P2, #0FFH
    MOV LED_P3, #0FFH
    CALL Update_595

; Infinite loop for 3 effects
MainLoop:
    CALL Eff1_Blink
    CALL Eff2_Heartbeat
    CALL Eff3_QuadrantChase
    JMP MainLoop

; EFFECT 1: BLINK (5s).
; Loop 5 times: ON 500ms, OFF 500ms

Eff1_Blink:
    MOV R5, #5    ; R5 = loop counter (5 times)
Loop_E1:
    ; Turn ON all LEDs (Logic 0 = ON)
    MOV LED_P0, #00H
    MOV LED_P1, #00H
    MOV LED_P2, #00H
    MOV LED_P3, #00H

```

```
CALL Update_595
MOV R4, #5      ; R4 = 5 -> Delay 500ms
CALL Delay_R4x100ms
```

```
; Turn OFF all LEDs (Logic 1 = OFF)
```

```
MOV LED_P0, #0FFH
MOV LED_P1, #0FFH
MOV LED_P2, #0FFH
MOV LED_P3, #0FFH
```

```
CALL Update_595
MOV R4, #5      ; Delay 500ms
CALL Delay_R4x100ms
```

```
DJNZ R5, Loop_E1 ; Repeat 5 times
RET
```

```
; EFFECT 2: HEARTBEAT (5s).
```

```
; Double beat: ON 100ms/OFF 100ms -> ON 100ms/OFF 700ms
```

```
Eff2_Heartbeat:
```

```
MOV R5, #5      ; Loop 5 beats
```

```
Loop_E2:
```

```
; Beat 1 (Fast)
```

```
MOV LED_P0, #00H
MOV LED_P1, #00H
MOV LED_P2, #00H
MOV LED_P3, #00H
```

```
CALL Update_595
MOV R4, #1      ; ON 100ms
CALL Delay_R4x100ms
```

```
MOV LED_P0, #0FFH
MOV LED_P1, #0FFH
MOV LED_P2, #0FFH
MOV LED_P3, #0FFH
```

```
CALL Update_595
MOV R4, #1      ; OFF 100ms
CALL Delay_R4x100ms
```

```

; Beat 2 (Slow)
MOV LED_P0, #00H
MOV LED_P1, #00H
MOV LED_P2, #00H
MOV LED_P3, #00H
CALL Update_595
MOV R4, #1      ; ON 100ms
CALL Delay_R4x100ms

MOV LED_P0, #0FFH
MOV LED_P1, #0FFH
MOV LED_P2, #0FFH
MOV LED_P3, #0FFH
CALL Update_595
MOV R4, #7      ; OFF 700ms (Rest period)
CALL Delay_R4x100ms

DJNZ R5, Loop_E2
RET

```

; EFFECT 3: QUADRANT CHASE (4.8s).

Eff3_QuadrantChase:

```

MOV R5, #6      ; Loop 6 rounds

```

Loop_E3:

```

; Step 1: ON P0, ensure others are OFF
MOV LED_P3, #0FFH ; Clear previous step (P3)
MOV LED_P0, #00H
CALL Update_595
MOV R4, #2        ; Delay 200ms
CALL Delay_R4x100ms

; Step 2: ON P1, turn OFF P0
MOV LED_P0, #0FFH
MOV LED_P1, #00H
CALL Update_595
MOV R4, #2

```

```
CALL Delay_R4x100ms
```

```
; Step 3: ON P2, turn OFF P1
```

```
MOV LED_P1, #0FFH
```

```
MOV LED_P2, #00H
```

```
CALL Update_595
```

```
MOV R4, #2
```

```
CALL Delay_R4x100ms
```

```
; Step 4: ON P3, turn OFF P2
```

```
MOV LED_P2, #0FFH
```

```
MOV LED_P3, #00H
```

```
CALL Update_595
```

```
MOV R4, #2
```

```
CALL Delay_R4x100ms
```

```
DJNZ R5, Loop_E3
```

```
RET
```

```
; 74HC595 DRIVERS
```

```
Update_595:
```

```
MOV A, LED_P3
```

```
CALL Shift_1_Byte
```

```
MOV A, LED_P2
```

```
CALL Shift_1_Byte
```

```
MOV A, LED_P1
```

```
CALL Shift_1_Byte
```

```
MOV A, LED_P0
```

```
CALL Shift_1_Byte
```

```
CLR ST_CP
```

```
SETB ST_CP
```

```
RET
```

```
Shift_1_Byte:
```

```
PUSH 02H
```

```
MOV R2, #8
```

```
Shift_Loop:
```

```

RLC A
MOV DS_PIN, C
CLR SH_CP
SETB SH_CP
DJNZ R2, Shift_Loop
POP 02H
RET

```

; REAL-TIME DELAY 12MHz (1 Machine Cycle = 1μs)

; Multiplier delay: Time = R4 x 100ms

Delay_R4x100ms:

```

    PUSH 04H      ; Save R4 to Stack to prevent corruption

```

D_R4: CALL Delay100ms

```

    DJNZ R4, D_R4
    POP 04H       ; Restore R4 original value
    RET

```

; Standard 100ms delay: $200 * (248 * 2 + 4) + 13 = 100,013$ cycles (~100ms)

Delay100ms:

```

    PUSH 06H      ; Save R6 to Stack
    PUSH 07H      ; Save R7 to Stack
    MOV R6, #200   ; Outer loop

```

D1: MOV R7, #248 ; Inner loop adjusted for 12MHz precision

D2: DJNZ R7, D2 ; 2 cycles per loop

```

    NOP           ; Timing compensation (1 cycle)
    DJNZ R6, D1    ; 2 cycles per loop
    POP 07H       ; Restore R7
    POP 06H       ; Restore R6
    RET

```

```

;=====

```

```

END

```

❖ Giải thích chi tiết:

Khai báo phân cứng & Bộ nhớ đệm (Virtual RAM):

```

SH_CP BIT P2.0
DS_PIN BIT P2.1
ST_CP BIT P2.2

```



```
LED_P0 DATA 30H
LED_P1 DATA 31H
LED_P2 DATA 32H
LED_P3 DATA 33H
```

- Mạch sử dụng 3 chân điều khiển thuộc Port 2 để giao tiếp với hệ thống IC dịch.
- Để tối ưu hóa quá trình xử lý, chương trình thiết lập 4 ô nhớ RAM (30H - 33H) làm bộ đệm dữ liệu (LED_P0 - LED_P3). Mọi thay đổi về trạng thái LED trong các hàm hiệu ứng sẽ được thực hiện trên các ô nhớ này trước khi gọi hàm xuất dữ liệu ra phần cứng.

Truyền dữ liệu mức Bit (Hàm Shift_1_Byte):

```
Shift_1_Byte:
    PUSH 02H
    MOV R2, #8
Shift_Loop:
    RLC A
    MOV DS_PIN, C
    CLR SH_CP
    SETB SH_CP
    DJNZ R2, Shift_Loop
    POP 02H
    RET
```

- Hàm thực hiện việc truyền 1 Byte dữ liệu từ thanh ghi A ra chân DS_PIN
- Lệnh RLC A dịch trái thanh ghi A qua cờ Carry, đưa bit có trọng số cao nhất (MSB) vào cờ C, sau đó lệnh MOV DS_PIN, C xuất giá trị này ra chân dữ liệu.
- Xung kích cạnh lên tại chân SH_CP (CLR rồi SETB) lệnh cho IC dịch bit hiện tại vào thanh ghi dịch nội bộ. Quá trình lặp lại 8 lần để hoàn tất 1 Byte.

Cơ chế cập nhật 32-bit & Chốt dữ liệu (Hàm Update_595):

```
Update_595:
    MOV A, LED_P3
    CALL Shift_1_Byte
    MOV A, LED_P2
    CALL Shift_1_Byte
    MOV A, LED_P1
    CALL Shift_1_Byte
    MOV A, LED_P0
    CALL Shift_1_Byte
```

CLR ST_CP

SETB ST_CP

RET

- **Truyền dữ liệu nối tầng:** Do 4 IC được mắc nối tiếp, byte dữ liệu được truyền vào đầu tiên sẽ bị đẩy dần sang các IC phía sau. Vì vậy, chương trình truyền dữ liệu từ LED_P3 (IC cuối cùng) đến LED_P0 (IC đầu tiên).
 - **Cơ chế chốt (Latching):** Sau khi đã nạp đủ 32 bit vào các Thanh ghi dịch (Shift Registers) của 4 IC, chương trình thực hiện tạo xung kích cạnh lên tại chân ST_CP. Thao tác này chuyển đồng loạt toàn bộ dữ liệu từ Thanh ghi dịch sang Thanh ghi lưu trữ (Storage Registers), giúp 32 ngõ ra cập nhật trạng thái cùng lúc, đảm bảo tín hiệu hiển thị ổn định, không bị nhấp nháy trong quá trình dịch bit.
2. Research and write the steps of printed circuits from the design step on the Proteus to the step of completing the printed circuit in practice.
- **Thiết kế sơ đồ (ISIS):** Vẽ sơ đồ nguyên lý trên Proteus, thực hiện kiểm tra lỗi kết nối (ERC) và gán đúng Footprint (gói linh kiện) thực tế cho từng linh kiện.
 - **Layout mạch in (ARES):** Chuyển sang môi trường ARES để sắp xếp linh kiện, thiết lập luật đi dây (DRC) về độ rộng đường mạch (Trace Width) và khoảng cách an toàn (Clearance).
 - **Phủ đồng & Xuất bản in:** Thực hiện đổ đồng (Copper Pour) để chống nhiễu và xuất file PDF lớp Bottom Copper ở chế độ Mirror để chuẩn bị in.
 - **In mạch (Printing):** Sử dụng máy in Laser và giấy in nhiệt (giấy bóng) để in sơ đồ mạch, mực Laser (nhựa) sẽ đóng vai trò là lớp mặt nạ bảo vệ đồng.
 - **Xử lý phôi đồng:** Dùng giấy nhám mịn và cồn/axeton vệ sinh sạch bóng bề mặt tấm phôi đồng để đảm bảo mực bám dính tốt nhất.
 - **Ủi nhiệt (Transferring):** Dùng bàn là hoặc máy ép nhiệt để ép mực từ giấy in bám chặt vào bề mặt đồng thông qua tác động của nhiệt độ cao.
 - **Ăn mòn hóa học (Etching):** Ngâm mạch vào dung dịch FeCl_3 hoặc $\text{HCl} + \text{H}_2\text{O}_2$ để hòa tan phần đồng thừa.
 - **Làm sạch & Khoan mạch:** Dùng xăng thơm/axeton tẩy sạch lớp mực đen, sau đó dùng máy khoan chuyên dụng (mũi 0.8mm - 1.2mm) để khoan lỗ chân linh kiện.
 - **Lắp ráp & Hàn (Soldering):** Phủ lớp nhựa thông loãng bảo vệ mạch, cắm linh kiện và thực hiện hàn chì, đảm bảo mối hàn bóng, đủ nhiệt và không chạm chập.
 - **Kiểm tra cuối cùng (Testing):** Sử dụng đồng hồ VOM đo thông mạch và ngắn mạch trước khi cấp nguồn vận hành thử nghiệm các hiệu ứng.

IV. References

- TS. Vũ Đức Lung, TS. Lê Quang Minh, ThS. Phan Đình Duy, *Giáo trình Vi điều khiển*, Trường Đại học Công nghệ Thông tin - ĐHQG TP.HCM, 2015
- Link video 1 (Content 1+2):
https://drive.google.com/file/d/1OBqmm6bgWEa78i4YsF3FRx8Lqaw_qm7B/view?usp=sharing
- Link video 2 (Exercise 1):
<https://drive.google.com/file/d/1gNtIZY5lZCYsILdYmWnN837PL1PZ9Y-/view?usp=sharing>